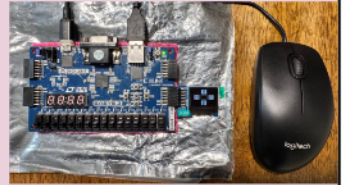
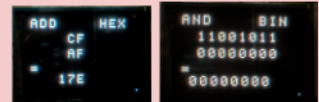
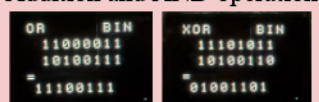
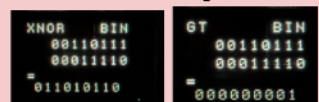
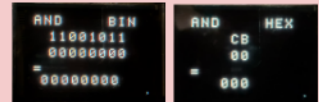
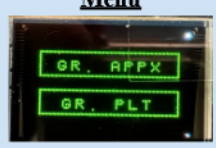
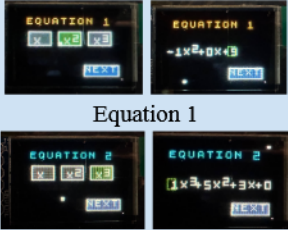
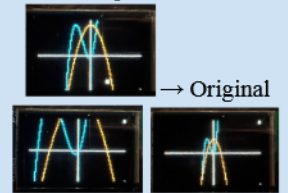
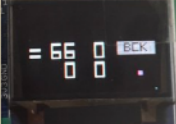
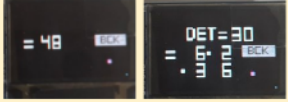
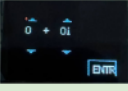


Members:

1. Aidan Chia Tong
2. Karthikeyan Vetrivel
3. Nicholas Lau Hongyi
4. Ryan Tan Rong Chang

Student and Component Name	Component Description	Images/Photos
<u>Team</u> "S3-11" <u>Project Title:</u> Multi-Modal FPGA-based GPMB Calculator (Matrix, Bit, Polynomial and Graphical)	Purpose: This is a mini app to help students solve various types of mathematical problems. Being multi-modal, we decided to make all of them different themes. We have a main menu, with each icon corresponding to a different theme: Mode <u>G</u>raph: Graph Approximation (Best Fit Line) and Graph Plotting Mode <u>M</u>atrix: Matrix Arithmetic, Determinant, Adjugate and Inverse of a 2x2 Matrix Mode <u>P</u>olynomial: Find real and imaginary roots, and plotting of the Argand Diagram Mode <u>B</u>it: Bit calculator and converter which can convert to Octal, Decimal and Hexadecimal, together with other operations as well General Navigation: You can navigate using a mouse. To open a selected theme, drag the respective square into the invisible box in the middle. To go back to the main menu, you can right click the mouse.	 Main Menu (Pmod OLED connected to Port JC and Mouse)  How to select mode
Student A: Aidan Chia Tong "Bit Calculator and Converter"	Controls: Down pushbutton to toggle between number systems (BIN, OCT, HEX), up pushbutton to toggle between the operations (AND, OR, XOR, ADD). Left pushbutton to change theme and right pushbutton to view history. Switches 0-15 are used to represent the bits at that position (OFF = 0, ON = 1). Center pushbutton to return to default settings (BIN and AND, in default colour). Bit addition: Performs 8-bit unsigned addition with 9-bit result, includes carry bit. Can be used for arithmetic operations, counter increment, etc. Bitwise AND: Performs bitwise logical AND operation. Can be used for bit masking, clearing specific bits, etc. Bitwise OR: Performs bitwise logical OR operation. Can be used for setting specific bits, combining bit patterns, etc. Bitwise XOR: Performs bitwise exclusive OR operation. Can be used for bit toggling, parity checking, simple encryption, etc. Bitwise XNOR: Performs bitwise logical XNOR operation. Can be used for equality checking, parity checking, etc. Greater than (GT): If A > B, return 1, else return 0. 8Bit BIN to OCT / HEX Converter: Use the switches 0-7 as input for the binary number. Use Down pushbutton to toggle between the number system to convert into OCT / HEX. *These features work under all number systems History: Shows last 2 calculated values in hexadecimal. Values are saved when mode is changed. Themes: Default white theme and matrix green theme	 Addition and AND operation  OR and XOR operation  XNOR and GT operation  Converting BIN to HEX  History mode and Matrix theme
Student B: Karthikeyan Vetrivel "Graphic Calculator and Best Fit Line"	Main Menu Page: Implemented a three-state FSM: IDLE → DRAGGING → DROPPED. When a box is dropped inside the dotted area, the corresponding theme is selected using a 4:1 multiplexer. Graph Approximation [Best Fit Plotting] In "Gr. Appx" mode, the user inputs three coordinates, each ranging from -9 to 9. Pushbuttons allow navigation and adjustment: PB UP / PB DOWN: Toggle current coordinate from -9 to 9. PB LEFT / PB RIGHT: Switch across different x and y coordinates. After entering the coordinates, the user clicks the "→" button to move to the next page, where the best-fit line is plotted using the formulas: $m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2} \quad \quad c = \frac{\sum y - m \sum x}{n}$ where m is the slope and c are the y-intercept. Calculations are spread across multiple clock cycles, controlled by an FSM	Menu  Gr. Appx  Coordinates  Best Fit Line

	that signals when slope and intercept computation is complete. Graph Plotting In "Gr. Plt" mode, users can plot two equations of type linear, quadratic, or cubic polynomials. The process is as follows: 1. Select polynomial type for equation 1 using mouse and click NEXT 2. Input coefficients for Equation 1 using PB. (-9 to 9) 3. Repeat the process for Equation 2 4. Finally, the two graphs are plotted on the display. Horner's method is used to minimise the number of multiplications to minimise LUT usage. [$y = ((ax + b)x + c)x + d$] The system also supports zoom in/out feature with a magnification factor of 8: Zoom in: Helps examine steep graphs and intercepts Zoom out: Provides a broader view, especially for larger cubic graphs. Users can return to the main menu at any time using the right mouse button. Both modes use fixed-point arithmetic, maximizing point resolution and avoiding truncation errors during calculations.	Gr. Plt  Equation 1 Equation 2  Original Zoom In Zoom out
Student C: Nicholas Lau Hongyi "Matrix Calculator"	Controls: This mode can be navigated via the buttons on the screen, which are all clicked using the left click of the mouse. Matrix Main Menu: You will be able to select 1 of 3 modes. (Arithmetic, Determinant/Adjugate, Inverse) If you want to return back to the Main Menu you can right click on the mouse. Arithmetic Mode: You can perform either Addition, Subtraction or Multiplication of 2 matrices, up to 2 digits per entry. Number pad and Edit Matrix: Each state will be using the same number pad and edit matrix. You can input numbers from -99 to +99. Once you have confirmed the entry, press 'OK' to move to the next number. To delete the current entry, press 'DEL'. To move to the next matrix/state, press 'NXT'. This applies to all modes. Random Number Generator: You can click on this button to randomly generate a number from -63 to +63. Applies for all the modes. Result Matrix Display: After you input the matrices and click on NXT, it will display the answer, ranging from -99 to +99. If the answer is greater than 2 digits, an 'X' will be displayed instead. This applies to all modes. Determinant/Adjugate: Calculates and displays the value of the determinant and adjugate of a single matrix up to 2 digits per entry. Inverse: Calculates and displays both the determinant and adjugate of a single matrix. The inverse is just $\det(A)/\text{adj}(A)$, where A is a 2x2 matrix. However, if $\det(A)=0$, there is no inverse, and the screen will display 'DET = 0' and '= X'.	 Main Menu and Submenu  Number Pad and Edit Matrix  Result Display for Arithmetic and Adjugate  Determinant Result and Inverse Result respectively
Student D: Ryan Tan Rong Chang "Polynomial Calculator and Argand Diagram"	Controls: This component is controlled entirely using the mouse. Left click to select the desired action, and right click anywhere on the screen to return back to the main menu. Menu Page: Use the mouse to select between polynomial mode or argand mode. Left clicking on the button will bring the user to the respective pages. Polynomial Mode: In this mode, the user will be shown a template of a quadratic equation in the format of $01x^2 + 00x + 00$, where the user clicks the corresponding up and down arrows to select the desired coefficient. This mode is able to calculate roots of equations up to degree 2. Quadratic and linear equations are supported. This calculator is able to support both real and complex roots. .Real Roots: After inputting an equation with real roots only (e.g $x^2 + 6x + 8$), click ENTR button, and the results page will render, showing the degree of the entered equation, and the roots r1 and r2. Results are rendered in hexadecimal format. Complex Roots: Input an equation that has complex roots (e.g $x^2 - 6x + 25$). After clicking the ENTR button, the results page renders, and shows the degree of the equation. For the complex roots, real and imaginary parts are displayed in hexadecimal, and the roots are labelled r1 and r2 respectively. Argand Mode: User interface is similar to polynomial mode, where the user adjusts the real and imaginary components using up and down by clicking on the arrows. After setting the desired equation, the user clicks on the ENTR button, and is brought to the results page. Argand Diagram Results: The y-axis represents the imaginary value, and x-axis represents real value. The system takes in the magnitude of the imaginary and real value, and plots the Argand Diagram.	 Menu Page and Polynomial mode user interface  Results page for real roots  Results page for complex roots  User interface for argand diagram mode.  Argand diagram rendered on the oled screen

Group ID: S3-11

Members:

1. Aidan Chia Tong A0307866R
2. Karthikeyan Vetrivel A0307459W
3. Nicholas Lau Hongyi A0308905B
4. Ryan Tan Rong Chang A0306841H

AI Declaration

ChatGPT was used to find alternative ways to store the numbers and letters in an attempt to reduce the LUT usage. Our group decided to use distributed RAM and block RAM. Below is an example:

```
// Using distributed ROM keeps it combinational (no pipeline).
(* rom_style = "distributed", ram_style = "distributed" *)
reg [4:0] FONT5x7 [0:128*7-1];

// ----- init only the glyphs you actually use -----
integer __fi;
initial begin
    for (__fi=0; __fi<128*7; __fi=__fi+1) FONT5x7[__fi] = 5'b00000;

    // Digits '0'..'9' (0x30..0x39) - simple bold-ish 5x7
    // 0
    FONT5x7[{7'h30,3'd0}] = 5'b11111; FONT5x7[{7'h30,3'd1}] = 5'b1000
    FONT5x7[{7'h30,3'd3}] = 5'b10001; FONT5x7[{7'h30,3'd4}] = 5'b1000
    FONT5x7[{7'h30,3'd6}] = 5'b11111;
```

ChatGPT was used to generate the 5 x 7 font table for numbers 0-9 and the necessary alphabets.

```
"1": get_char = {7'b0000000, 7'b1000010, 7'b1111111, 7'b1000000, 7'b0000000};
"2": get_char = {7'b0110010, 7'b1001001, 7'b1001001, 7'b1001001, 7'b1000110};
"3": get_char = {7'b0100010, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0110110};
"4": get_char = {7'b0000111, 7'b0001000, 7'b0001000, 7'b0001000, 7'b1111111};
"5": get_char = {7'b0100111, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0110001};
"6": get_char = {7'b0111110, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0110010};
"7": get_char = {7'b0000001, 7'b1110001, 7'b0001001, 7'b0000101, 7'b0000011};
"8": get_char = {7'b0110110, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0110110};
"9": get_char = {7'b0100110, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0111110};
"A": get_char = {7'b1111110, 7'b0001001, 7'b0001001, 7'b0001001, 7'b1111110};
"B": get_char = {7'b1111111, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0110110};
"C": get_char = {7'b0111110, 7'b1000001, 7'b1000001, 7'b1000001, 7'b0100010};
"D": get_char = {7'b1111111, 7'b1000001, 7'b1000001, 7'b1000001, 7'b0111110};
"E": get_char = {7'b1111111, 7'b1001001, 7'b1001001, 7'b1001001, 7'b1000001};
"F": get_char = {7'b1111111, 7'b0001001, 7'b0001001, 7'b0001001, 7'b0000001};
"G": get_char = {7'b0111110, 7'b1000001, 7'b1001001, 7'b1001001, 7'b0110101};
"H": get_char = {7'b1111111, 7'b0001000, 7'b0001000, 7'b0001000, 7'b1111111};
"I": get_char = {7'b0000000, 7'b1000001, 7'b1111111, 7'b1000001, 7'b0000000};
"L": get_char = {7'b1111111, 7'b1000000, 7'b1000000, 7'b1000000, 7'b1000000};
"M": get_char = {7'b1111111, 7'b0000010, 7'b0001100, 7'b0000010, 7'b1111111};
"N": get_char = {7'b1111111, 7'b0000100, 7'b0001000, 7'b0010000, 7'b1111111};
"O": get_char = {7'b0111110, 7'b1000001, 7'b1000001, 7'b1000001, 7'b0111110};
"R": get_char = {7'b1111111, 7'b0001001, 7'b0001001, 7'b0001001, 7'b1110110};
"S": get_char = {7'b0100110, 7'b1001001, 7'b1001001, 7'b1001001, 7'b0110010};
"T": get_char = {7'b0000001, 7'b0000001, 7'b1111111, 7'b0000001, 7'b0000001};
"X": get_char = {7'b1100011, 7'b0010100, 7'b0001000, 7'b0010100, 7'b1100011};
">": get_char = {7'b1000001, 7'b0100010, 7'b0010100, 7'b0001000, 7'b0000000};
"<": get_char = {7'b0001000, 7'b0010100, 7'b0100010, 7'b1000001, 7'b0000000};
"=": get_char = {7'b0010100, 7'b0010100, 7'b0010100, 7'b0010100, 7'b0010100};
"." : get_char = {7'b0000000, 7'b0000000, 7'b0100100, 7'b0000000, 7'b0000000};
"-": get_char = {7'b0001000, 7'b0001000, 7'b0001000, 7'b0001000, 7'b0001000};
default: get_char = {7'b0000000, 7'b0000000, 7'b0000000, 7'b0000000, 7'b0000000};
```

Other References Used:

For pseudo-random number generator, formula for max-length LFSR taken from <https://docs.amd.com/v/u/en-US/xapp052>

For the generation of the letters on oled, Pillow library, accompanied by a python script was used.

<https://github.com/python-pillow/Pillow>

Course related feedback:

Nicholas: The labs could have touched on how to utilise things like BRAM, or the theory behind LUT or IO usage which are the two highest utilised components, so that we are able to optimise our FPGA better.

Vetri: While the tasks PQRS were quite useful as an introduction, I felt that the OLED could have been introduced early on in the Labs 1, 2, 3 with tasks such as implementing a FSM on the OLED module or simple exercises like clicking / dragging a button using a mouse.

Aidan: Introduction to OLED screens earlier would have been helpful for the project, as learning to integrate the OLED into our work with limited background knowledge was challenging.

Ryan: Concepts like functions could have been taught as some of us ended up using them in place of modules to decrease LUT usage.